🏢 Edulateral Foundation — Official Evaluative Assessment Bank

Day 3: Deep Learning on HPC with TensorFlow, Slurm & Hardware Accelerators

Target Audience: GLA University 7th Semester B.Tech Computer Science & Engineering
Lead Trainer: Shivam Gupta | **Evaluation Mode**: Objective Technical MCQ Assessment
Total Questions: 25 Questions | **Time Limit**: 45 Minutes

Question 01: D3-Q01

What is the primary role of Intel oneDNN (oneAPI Deep Neural Network Library) in deep learning frameworks like TensorFlow and PyTorch?

- **(A)** It acts as a graphical user interface for training neural networks.
- **(B)** It provides highly optimized compute kernels (convolution, GEMM, pooling, normalization, attention) with automatic operator fusion and JIT-compiled assembly for Intel Xeon CPUs and GPUs.
- **(C)** It is a cloud storage database for dataset images.
- **(D)** It automatically generates Python code from natural language prompts.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Intel oneDNN contains hand-tuned micro-kernels for deep learning primitives. It dynamically JIT-compiles assembly code targeting the host CPU's instruction set (AVX-512, AMX, VNNI) and performs operator fusion (e.g. Conv + Bias + ReLU in a single cache-resident pass).

Question 02: D3-Q02

What environment variable must be set in TensorFlow to ensure oneDNN optimizations are fully enabled on Intel Xeon processors?

- **(A)** TF_ENABLE_ONEDNN_OPTS=1
- **(B)** TF_DISABLE_INTEL=1
- **(C)** CUDA_VISIBLE_DEVICES=-1
- **(D)** OMP_NUM_THREADS=0

> **Correct Answer**: **(A)**
> **Technical Explanation**: Setting `TF_ENABLE_ONEDNN_OPTS=1` activates TensorFlow's oneDNN operator graph rewrite pass, replacing default Eigen GEMM and Conv operators with optimized Intel oneDNN primitives.

Question 03: D3-Q03

What are the two distinct thread pools managed by TensorFlow on multi-core HPC servers, and how do their functions differ?

- **(A)** Fast threads and Slow threads.
- **(B)** intra_op_parallelism_threads (parallelizes individual tensor ops like matrix multiplications across cores) and inter_op_parallelism_threads (executes independent nodes in the computation graph concurrently).
- **(C)** Read threads and Write threads.
- **(D)** GPU threads and RAM threads.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `intra_op_parallelism_threads` controls how many threads work together to execute a single compute-intensive operation (e.g. multi-threaded MatMul). `inter_op_parallelism_threads` dictates how many independent operations in the execution graph run in parallel.

Question 04: D3-Q04

In the Slurm Workload Manager, what daemon runs on every compute node to monitor local hardware resources and execute allocated tasks?

- **(A)** slurmctld
- **(B)** slurmd
- **(C)** slurmdbd
- **(D)** systemd

> **Correct Answer**: **(B)**
> **Technical Explanation**: `slurmd` is the node compute daemon that runs on every compute node. It listens for job launch requests from the central controller (`slurmctld`), spawns tasks, manages cgroups, and reports node health.

Question 05: D3-Q05

What is the function of the central `slurmctld` daemon in an HPC cluster?

- **(A)** It compiles C++ source code.
- **(B)** It acts as the central management controller daemon that monitors node states, evaluates job queues, applies priority policies, and schedules jobs across compute nodes.
- **(C)** It controls room air conditioning.
- **(D)** It powers off the user's laptop.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `slurmctld` (Slurm Controller Daemon) is the orchestrator of the entire cluster. It maintains cluster state, schedules pending jobs according to fairness and resource constraints, and directs compute node daemons.

Question 06: D3-Q06

Which Slurm command is used to submit an asynchronous batch job script containing `#SBATCH` directives to the scheduling queue?

- **(A)** srun

- **(B)** sbatch
- **(C)** salloc
- **(D)** squeue

> **Correct Answer**: **(B)**
> **Technical Explanation**: `sbatch` submits a batch script to the Slurm queue. It parses `#SBATCH` directives, registers the job in the pending queue, and returns the assigned Job ID immediately, allowing the user to disconnect while the script executes asynchronously.

Question 07: D3-Q07

In a Slurm batch script, what does the directive `#SBATCH --nodes=4 --ntasks-per-node=2 --cpus-per-task=16` allocate?

- **(A)** 4 nodes, each running 2 tasks (8 total MPI ranks/processes), with each task allocated 16 CPU cores (128 total CPU cores across cluster).
- **(B)** 4 CPU cores in total.
- **(C)** 4 gigabytes of RAM across 2 hard drives.
- **(D)** 16 GPUs on 1 computer.

> **Correct Answer**: **(A)**
> **Technical Explanation**: `--nodes=4` reserves 4 physical server nodes; `--ntasks-per-node=2` launches 2 processes per node ($4 \times 2 = 8$ tasks total); and `--cpus-per-task=16` assigns 16 hardware CPU cores to each task ($8 \times 16 = 128$ cores total), ideal for hybrid MPI + OpenMP applications.

Question 08: D3-Q08

What is the fundamental algorithmic difference between the Parameter Server architecture and Horovod's Ring-AllReduce architecture for distributed deep learning?

- **(A)** Parameter Server uses GPUs; Horovod only uses floppy disks.
- **(B)** Parameter Server suffers from communication bottlenecks because all workers send gradients to central server nodes; Ring-AllReduce organizes workers in a logical ring where communication cost is constant and independent of the number of nodes.
- **(C)** Horovod sends gradients via email.
- **(D)** Parameter Server trains 100x faster than Ring-AllReduce on 1,000 GPUs.

> **Correct Answer**: **(B)**
> **Technical Explanation**: In Parameter Server architectures, network bandwidth at the parameter server scales linearly with worker count $O(N)$, causing severe saturation. In Ring-AllReduce (Baidu/Horovod algorithm), each node only communicates with its immediate neighbors in a ring, transferring a constant volume of data $2\frac{N-1}{N}M$ that asymptotically approaches $2M$, scaling efficiently to thousands of nodes.

Question 09: D3-Q09

In Horovod, why must the learning rate typically be scaled proportionally to the number of distributed workers (`hvd.size()`)?

- **(A)** Because CPU clock speed increases proportionally with node count.
- **(B)** Because distributed data parallelism increases the effective global batch size by a factor of `hvd.size()`, requiring a proportionally larger learning rate (Linear Scaling Rule) to maintain gradient descent step dynamics.
- **(C)** Because Horovod divides the model parameters by `hvd.size()`.
- **(D)** Because Slurm will reject jobs with small learning rates.

> **Correct Answer**: **(B)**
> **Technical Explanation**: According to the Linear Scaling Rule (Goyal et al., 2017), when training with K workers, the effective batch size is $K \times B$. To ensure the stochastic gradient step covers equivalent parameter distance relative to single-worker training, the base learning rate must be scaled by K (accompanied by learning rate warmup).

Question 10: D3-Q10

In a Horovod distributed training script, which rank is traditionally designated to save model checkpoints and log training metrics to disk to prevent file corruption?

- **(A)** Rank 0 (`hvd.rank() == 0`)
- **(B)** All ranks simultaneously
- **(C)** The last rank (`hvd.rank() == hvd.size() - 1`)
- **(D)** Rank -1

> **Correct Answer**: **(A)**
> **Technical Explanation**: Rank 0 is the master rank. If all ranks attempt to write checkpoint files to the same shared parallel file system path simultaneously, file lock collisions and race conditions will corrupt the checkpoint. Guarding disk I/O with `if hvd.rank() == 0:` ensures only one process writes to disk.

Question 11: D3-Q11

What is the purpose of Horovod's `hvd.BroadcastGlobalVariablesCallback(0)` callback in TensorFlow/Keras?

- **(A)** It broadcasts the model architecture to student smartphones.
- **(B)** It broadcasts initial random weights and optimizer states from Rank 0 to all other worker ranks at epoch 0, ensuring all workers start with identical model parameters.
- **(C)** It prints the learning rate on the terminal.
- **(D)** It deletes previous model checkpoints.

> **Correct Answer**: **(B)**
> **Technical Explanation**: If each worker initializes its weights randomly, their gradients will be calculated from different starting points, preventing convergence. `BroadcastGlobalVariablesCallback(0)` synchronizes the exact initial weights from Rank 0 across all workers before training begins.

Question 12: D3-Q12

In Habana Gaudi (Gaudi 2 / Gaudi 3) deep learning processors, what dedicated on-chip networking hardware replaces standard PCIe network cards?

- **(A)** Standard USB 2.0 ports.
- **(B)** 24 integrated 100-Gigabit Ethernet (RoCE v2) ports embedded directly on each Gaudi silicon die for all-to-all scale-out without external host NICs.
- **(C)** VGA video output cables.
- **(D)** Wi-Fi antennas.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Habana Gaudi silicon integrates twenty-four 100GbE RoCE v2 (RDMA over Converged Ethernet) ports directly on-chip. This enables direct chip-to-chip interconnectivity across accelerators and racks without traversing host PCIe buses or requiring expensive external NICs.

Question 13: D3-Q13

What are the primary compute engines inside Habana Gaudi processors responsible for tensor contractions?

- **(A)** Matrix Multiplication Engines (MME) paired with Tensor Processing Cores (TPC).
- **(B)** Sound Blaster audio processors.
- **(C)** x86 scalar arithmetic units.
- **(D)** Floppy disk controllers.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Habana Gaudi features specialized Matrix Multiplication Engines (MME) tailored for massive GEMM operations and programmable VLIW Tensor Processing Cores (TPC) for element-wise and non-linear tensor activations.

Question 14: D3-Q14

When loading training datasets in a distributed Horovod pipeline, what method ensures that each worker receives a unique partition of the training samples?

- **(A)** dataset.shard(num_shards=hvd.size(), index=hvd.rank())
- **(B)** dataset.delete()
- **(C)** dataset.repeat(100)
- **(D)** dataset.shuffle_all_ranks()

> **Correct Answer**: **(A)**
> **Technical Explanation**: `dataset.shard(num_shards=hvd.size(), index=hvd.rank())` divides the dataset into `hvd.size()` equal parts and assigns partition `hvd.rank()` to the local worker, preventing redundant duplicate training.

Question 15: D3-Q15

What Slurm command displays real-time status of all active, pending, and running jobs across the cluster?

- **(A)** squeue
- **(B)** scancel
- **(C)** scontrol reboot
- **(D)** top

> **Correct Answer**: **(A)**
> **Technical Explanation**: `squeue` queries the Slurm controller daemon, displaying active jobs, Job IDs, partitions, job names, user accounts, current execution states (RUNNING, PENDING, COMPLETING), elapsed runtimes, and allocated nodes.

Question 16: D3-Q16

What Slurm command is used to terminate a running or queued job with Job ID 12345?

- **(A)** scancel 12345
- **(B)** sbatch --delete 12345
- **(C)** srun --kill 12345
- **(D)** slurm exit 12345

> **Correct Answer**: **(A)**
> **Technical Explanation**: `scancel <job_id>` signals `slurmctld` to abort the specified job, sending `SIGTERM` followed by `SIGKILL` to running processes and releasing allocated compute nodes back to the cluster pool.

Question 17: D3-Q17

In Slurm cluster scheduling, what does the state 'PD' (PENDING) with reason 'Priority' signify?

- **(A)** The job failed with a syntax error.
- **(B)** The job is waiting in the scheduling queue because higher-priority jobs are ahead of it in the multifactor priority queue.
- **(C)** The compute nodes are currently powered off.
- **(D)** The user forgot to submit their password.

> **Correct Answer**: **(B)**
> **Technical Explanation**: In Slurm's multifactor priority plugin, jobs receive a composite priority based on job age, fair-share allocation, partition, and resource requests. 'Resources' or 'Priority' indicates the job is queued waiting for its turn or for required nodes to become free.

Question 18: D3-Q18

What precision format uses 1 sign bit, 8 exponent bits, and 7 mantissa bits, matching the dynamic range of FP32 while requiring only 16 bits of memory storage?

- **(A)** FP16 (IEEE half precision)

- **(B)** Bfloat16 (Brain Floating Point)
- **(C)** INT8
- **(D)** TF32

> **Correct Answer**: **(B)**
> **Technical Explanation**: Bfloat16 allocates 8 bits to the exponent (identical to FP32), giving it the same dynamic range as FP32 and preventing underflow/overflow during deep learning backpropagation, while reducing memory bandwidth consumption by 50%.

Question 19: D3-Q19

How does Intel AMX (Advanced Matrix Extensions) accelerate Bfloat16 matrix multiplication in 4th/5th Gen Intel Xeon processors?

- **(A)** By calculating matrices on an external USB graphics card.
- **(B)** By loading matrices into 2D tile registers (TMM) and executing hardware matrix multiplication in the Tile Matrix Multiply (TMUL) unit in a single instruction.
- **(C)** By converting all floats into integers.
- **(D)** By offloading matrix calculations to the network switch.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Intel AMX introduces 2D tile registers (TMM0-TMM7) capable of holding sub-matrices. The TMUL hardware accelerator computes matrix product operations (e.g. $C+ = A \times B$) directly on BF16 or INT8 tiles, executing over 1000 operations per cycle.

Question 20: D3-Q20

What is the role of Horovod's Tensor Fusion buffer in distributed deep learning?

- **(A)** It fuses multiple small gradient tensors into a single contiguous ring-AllReduce buffer to maximize network throughput and minimize communication latency.
- **(B)** It fuses the GPU silicon to prevent overheating.
- **(C)** It compresses training images into JPEG format.
- **(D)** It deletes redundant layers from the neural network.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Transmitting thousands of tiny gradient tensors across the network incurs excessive latency overhead per message. Horovod's Tensor Fusion batches small gradients into a configurable ring buffer (e.g. 64 MB) before invoking AllReduce, maximizing network saturation.

Question 21: D3-Q21

Which Slurm command allows an engineer to request an interactive bash shell session on a compute node for real-time debugging?

- **(A)** salloc --nodes=1 --cpus-per-task=8 --pty bash (or srun --pty bash)
- **(B)** sbatch debug.sh

- **(C)** ssh root@supercomputer
- **(D)** scontrol restart

> **Correct Answer**: **(A)**
> **Technical Explanation**: `salloc` (or `srun --pty bash`) requests an interactive allocation from Slurm. Once resources are assigned, Slurm connects standard input/output directly to a terminal shell on the compute node, enabling interactive profiling and debugging.

Question 22: D3-Q22

What memory allocator is widely used in high-performance TensorFlow deployments on Intel Xeon servers to eliminate multi-threaded heap contention compared to default glibc malloc?

- **(A)** tcmalloc (Thread-Caching Malloc) or jemalloc
- **(B)** DOS mem.exe
- **(C)** Windows Pagefile allocator
- **(D)** None; dynamic memory is prohibited.

> **Correct Answer**: **(A)**
> **Technical Explanation**: Standard glibc malloc uses global locks that suffer severe lock contention when hundreds of threads concurrently allocate tensor buffers. Google's `tcmalloc` or `jemalloc` implement per-thread memory caches, providing lock-free allocations and eliminating memory fragmentation.

Question 23: D3-Q23

What does the Slurm directive `#SBATCH --time=02:00:00` specify?

- **(A)** The job must start exactly at 2:00 AM.
- **(B)** The maximum wall-clock time limit for the job (2 hours); if execution exceeds this limit, Slurm terminates the job.
- **(C)** The job will take 2 seconds to run.
- **(D)** The job repeats every 2 hours indefinitely.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `--time=HH:MM:SS` sets the maximum walltime limit for the job. Accurate walltime estimates help Slurm's backfill scheduler slot smaller jobs into scheduling gaps, improving overall cluster utilization.

Question 24: D3-Q24

What is the primary benefit of operator fusion (e.g., fusing Convolution + BiasAdd + ReLU) in deep learning frameworks optimized with oneDNN?

- **(A)** It converts 2D convolutions into 1D convolutions.
- **(B)** It keeps intermediate tensor activation data in ultra-fast L1/L2 CPU cache memory, eliminating round-trip writes and reads to slow main DRAM.
- **(C)** It reduces the number of training labels.
- **(D)** It trains the model without backpropagation.

> **Correct Answer**: **(B)**
> **Technical Explanation**: In standard execution, Convolution writes its output to main DRAM, BiasAdd reads it back and writes to DRAM, and ReLU reads and writes to DRAM. Operator fusion combines these three operations into a single kernel loop, keeping intermediate values in registers/L1 cache and eliminating DRAM memory bandwidth bottlenecks.

Question 25: D3-Q25

What does `sinfo` command display on an HPC cluster?

- **(A)** The system sound volume.
- **(B)** Summary of cluster partitions, node availability, socket states (idle, alloc, down), and node limits.
- **(C)** The system clock time.
- **(D)** List of installed Python packages.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `sinfo` queries the Slurm cluster partition and node tables, displaying partition names, default states, node counts, and the current operational states of nodes (idle, allocated, completing, drained, or down).
